Define user defined packages. Steps in creating and using packages.(15 marks)

User-Defined Packages in Java:

In Java, packages are used to group related classes, interfaces, and sub-packages. They help in organizing the code, avoiding name conflicts, and improving code modularity and maintainability. A **user-defined package** is a package that is created by the programmer (developer) to store related classes and interfaces. Java provides two types of packages: **built-in packages** (e.g., java.util, java.io) and **user-defined packages** (created by users to organize their code).

Benefits of Using User-Defined Packages:

1. **Code Organization:** Packages help in logically grouping classes and interfaces, which makes the code easier to maintain.
2. **Namespace Management:** Packages provide a namespace mechanism to avoid class name conflicts.
3. **Access Protection:** By using packages, access modifiers can be applied to restrict access to certain classes or interfaces.
4. **Code Reusability:** Code can be reused more effectively when grouped in packages.

Steps to Create and Use a User-Defined Package:

1. Creating a User-Defined Package:

To create a user-defined package in Java, follow these steps:

Step 1: Choose the name of the package. By convention, package names are written in lowercase letters.

Step 2: Create a Java file and begin the code with the package keyword, followed by the package name. The package statement must be the first line in the Java source file.

Example:

```
package mypackage;  // This defines the package name
public class MyClass {
   public void display() {
      System.out.println("Hello from MyClass in mypackage!");
   }
}
```

In this example, the class MyClass is defined under the package mypackage.

2. Saving the Java File:

The Java file should be saved with the .java extension in the corresponding directory that matches the package name. For example, if the package is mypackage, the file should be saved in a folder named mypackage.

File structure:

```
mypackage/
   MyClass.java
```

3. Compiling the Java Class:

To compile the Java class in a user-defined package, use the javac command. You must run the command from the directory above mypackage.

```
javac mypackage/MyClass.java
```

This will compile the class MyClass and create a MyClass.class file in the mypackage folder.

4. Using a User-Defined Package:

Once the package is created and compiled, you can use it in other Java programs by importing the package.

Step 1: Import the package in your program using the import keyword.

Step 2: Use the class defined in the package as required.

Example:

```
import mypackage.MyClass;  // Importing the user-defined package

public class Test {
   public static void main(String[] args) {
      MyClass obj = new MyClass();  // Creating an object of MyClass from
mypackage
      obj.display();  // Calling the display method
   }
}
```

In this example, the Test class imports the MyClass class from the mypackage package and uses its display method.

5. Running the Program:

After compiling the classes, you can run the program by using the java command. Make sure that the classpath includes the directory where the package is stored.

```
java Test
```

Output:

```
Hello from MyClass in mypackage!
```

6. Package Hierarchy (Optional):

Java also supports nested or hierarchical packages. For example, you can have packages within packages, like mypackage.subpackage. In this case, the directory structure would reflect the package hierarchy.

To create a nested package:

```
package mypackage.subpackage;

public class SubClass {
    public void display() {
        System.out.println("Hello from SubClass in mypackage.subpackage!");
    }
}
```

To use it:

```
import mypackage.subpackage.SubClass;

public class Test {
    public static void main(String[] args) {
        SubClass obj = new SubClass();
        obj.display();
    }
}
```

Conclusion:

User-defined packages in Java provide a way to organize and group related classes and interfaces. By using the package and import keywords, developers can efficiently manage large projects, ensure code reuse, and avoid naming conflicts. Creating packages is a fundamental concept in Java programming that helps structure the code logically.